

```
span("checkout"):
  process_payment()
  update_inventory()
```

In real production environments, tracing often exposes weird or unexpected bottlenecks. An e-commerce platform may notice that checkout feels slow not because of payment processing, but because an external recommendation service adds extra latency.

That kind of clarity matters a lot in microservices setups, where dependencies are often hidden and can change over time. Tracing lets engineers replay the request pathway and spot inefficiencies that would have otherwise stayed invisible.

As systems get more connected and interdependent, tracing acts like the connecting bridge that links metrics and logs into one investigative view.

The observability control centre: Visualising everything with Grafana

Raw telemetry data turns into something meaningful only when it can be interpreted in a way that helps people. Visualisation platforms matter a lot here, because they turn those tangled datasets into operational views that are easier to grasp, even when things get messy.

Grafana has become one of the more adopted tools for this purpose. It connects with several data sources so teams can show metrics, logs, and traces together in one place — like a shared screen for everyone.

When the dashboard is built well, it gives near real time understanding of system health, pulling together infrastructure, applications, and business indicators in a single view.

```
dashboard:
  panels:
    - CPU Usage
    - Request Latency
    - Error Rate
    - Revenue Per Minute
```

In the real world, these kinds of dashboards show up a lot during busy moments like online sales, or when a product launch goes live. Engineering teams keep an eye on system behaviour as it happens, while product and business folks watch user engagement and revenue impact side by side.

Grafana also supports cross teamwork because it gives people a shared operational vocabulary. Rather than struggling with raw logs, or staring at isolated metrics, everyone can work from one common visual context. So as visualisation becomes the core of day-to-day choices, observability shifts from being just a backend engineering concern into more of an organisation-wide skillset.

Observability for cloud-native, AI, and edge workloads

Cloud-native setups, AI models, and edge devices add extra layers of complexity that the older observability methods struggle to map correctly. Honestly, it's messier than people expect.

Cloud-native systems are very dynamic, with containers scaling up or down within seconds, and sometimes it feels like everything is moving at once. So observability tools need to adapt in real time, not just when something is already broken.

AI systems bring their own set of headaches. Besides usual infrastructure metrics, teams also need to watch model performance, inference latency, and prediction quality.

```
{
  "model": "fraud-detection-v3",
  "inference_ms": 38,
  "confidence": 0.91
  "drift_score": 0.12
}
```

Edge environments make observability even more awkward. Limited connectivity, plus distributed deployment locations, means telemetry

must be handled carefully. Sensors, IoT systems, and industrial controllers must send data efficiently, but not flood the networks, or at least not all at once.

In fields like manufacturing and logistics, observability is what helps physical and digital components stay reliable. If sensor readings come in late or model inference drifts, it can cause real world operational trouble, even if the dashboards look “mostly fine” for a minute.

So as computing keeps decentralising, observability must evolve to deliver consistent visibility across these different, heterogeneous environments.

Building a production-ready open source observability platform

Building a production-grade observability system means way more than dropping in tools. It needs architectural choices that juggle scalability, reliability, and cost efficiency all at the same time.

In most real deployments the telemetry pathway gets split into a few layers — ingestion, processing, storage and, finally, visualisation. That separation lets each part scale on its own without dragging the rest of the stack along with it.

Apps → OpenTelemetry → processing pipeline → Storage (Metrics Logs Traces) → Grafana

At the organisational level, teams also have to think about retention periods -- who can access what, and whether there is high availability. Keeping every single telemetry signal forever is not practical, nor is it usually budget-friendly. So systems focus on high-value data while archiving or sampling the less crucial information.

■ ■ Continued to page...51